

KUBERNETES - 06

1. Create a ConfigMap from a directory containing multiple files and inject the variables into a pod as environment variables.

Step 1: Create directory

```
mkdir config
```

Step 2: Create first file

```
ubuntu@k8s-master:~$ mkdir config
ubuntu@k8s-master:~$ cat > config/app.properties <<EOF
> APP_NAME=Myapplications
> APP_ENV=DEV
EOF
ubuntu@k8s-master:~$ cat config/app.properties
APP_NAME=Myapplications
APP_ENV=DEV
ubuntu@k8s-master:~$
```

Step 3: Create second file

```
ubuntu@k8s-master:~$ cat > config/database.properties <<EOF
DB_HOST=mysql
DB_PORT=3306
EOF
ubuntu@k8s-master:~$ cat config/database.properties
DB_HOST=mysql
DB_PORT=3306
ubuntu@k8s-master:~$
```

Step 4: Create ConfigMap

```
kubectl create configmap app-config --from-file=config/
```

```
ubuntu@k8s-master:~$ kubectl create configmap app-config --from-file=config/
configmap/app-config created
ubuntu@k8s-master:~$ kubectl get configmap
NAME          DATA  AGE
app-config    2      11s
kube-root-ca.crt 1      8d
```

Check details:

```
ubuntu@k8s-master:~$ kubectl describe configmap app-config
Name:         app-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
app.properties:
-----
APP_NAME=Myapplications
APP_ENV=DEV

database.properties:
-----
DB_HOST=mysql
DB_PORT=3306

BinaryData
=====
Events:      <none>
ubuntu@k8s-master:~$
```

Here the ConfigMap contains **2 keys**:

app.properties

database.properties

Step 5: Create Pod

```
ubuntu@k8s-master:~$ vi configmap-env-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f configmap-env-pod.yaml
pod/configmap-env-pod created
```

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-env-pod
spec:
  containers:
  - name: nginx
    image: nginx
    envFrom:
    - configMapRef:
        name: app-config
```

Step 6: Verify

kubectl get pod configmap-env-pod

Enter the Pod:

kubectl exec -it configmap-env-pod -- /bin/bash

Check:

env

You should find:

```
ubuntu@k8s-master:~$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
configmap-env-pod   1/1     Running   0           10s
default-nginx       1/1     Running   1 (129m ago) 25h
ubuntu@k8s-master:~$ kubectl exec -it configmap-env-pod -- /bin/bash
root@configmap-env-pod:/# env
database.properties=DB_HOST=mysql
DB_PORT=3306

app.properties=APP_NAME=Myapplications
APP_ENV=DEV

KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_PORT=443
NGINX_SERVICE_PORT_80_TCP_ADDR=10.107.191.109
HOSTNAME=configmap-env-pod
NGINX_SERVICE_PORT_80_TCP=tcp://10.107.191.109:80
NGINX_SERVICE_SERVICE_HOST=10.107.191.109
PWD=/
ACME_VERSION=0.4.1
NGINX_SERVICE_PORT_80_TCP_PROTO=tcp
PKG_RELEASE=1~trixie
HOME=/root
```

Result

ConfigMap values are available as **environment variables**.

2. Create a ConfigMap from a file and mount it as a volume inside a pod, ensuring the configuration data is available as files.

Requirement

Create a ConfigMap from a file and mount it inside a Pod.

The configuration should appear as a file inside the container

Step 1: Create configuration file

```
ubuntu@k8s-master:~$ cat > application.conf <<EOF
server.port=8080
server.name=my-app
environment=dev
EOF
ubuntu@k8s-master:~$ cat application.conf
server.port=8080
server.name=my-app
environment=dev
ubuntu@k8s-master:~$
```

Step 2: Create ConfigMap

```
ubuntu@k8s-master:~$ kubectl create configmap application-config \
  --from-file=application.conf
configmap/application-config created
ubuntu@k8s-master:~$ kubectl get configmap application-config
NAME          DATA   AGE
application-config  1      13s
ubuntu@k8s-master:~$
```

Step 3: Create Pod YAML & apply

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-volume-pod
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
        - name: config-volume
          mountPath: /etc/myapp
  volumes:
    - name: config-volume
      configMap:
        name: application-config
```

```
ubuntu@k8s-master:~$ vi configmap-volume-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f configmap-volume-pod.yaml
pod/configmap-volume-pod created
ubuntu@k8s-master:~$
```

Step 5: Enter the Pod and Check inside pod

```
ubuntu@k8s-master:~$ kubectl exec -it configmap-volume-pod -- /bin/bash
root@configmap-volume-pod:/# ls /etc/myapp
application.conf
root@configmap-volume-pod:/# cat /etc/myapp/application.conf
server.port=8080
server.name=my-app
environment=dev
root@configmap-volume-pod:/#
```

Understand the flow:

application.conf



ConfigMap



Volume



Pod /etc/myapp/



application.conf

3. Create a Secret with sensitive information (username and password) and inject it into a pod as environment variables.

Step 1: Create Secret

```
ubuntu@k8s-master:~$ kubectl create secret generic db-secret \
> --from-literal=username=admin \
> --from-literal=password=Mypass123
secret/db-secret created
ubuntu@k8s-master:~$ kubectl get secret
NAME          TYPE          DATA   AGE
db-secret     Opaque        2       18s
ubuntu@k8s-master:~$
```

Step 2: Create Pod

```
ubuntu@k8s-master:~$ vi secret-env-pod.yaml
ubuntu@k8s-master:~$
```

```

apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
spec:
  containers:
    - name: nginx
      image: nginx

      env:
        - name: DB_USERNAME
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: username

        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: password

```

Step 3: Apply & Check pods

```

ubuntu@k8s-master:~$ kubectl apply -f secret-env-pod.yaml
pod/secret-env-pod created
ubuntu@k8s-master:~$

```

```

ubuntu@k8s-master:~$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
configmap-env-pod   1/1     Running   0           32m
configmap-volume-pod 1/1     Running   0           15m
default-nginx       1/1     Running   1 (161m ago) 25h
secret-env-pod      1/1     Running   0           69s
ubuntu@k8s-master:~$

```

Step 4: Check inside Pod & Check output

```

ubuntu@k8s-master:~$ kubectl exec -it secret-env-pod -- /bin/bash
root@secret-env-pod:/# echo $DB_USERNAME
admin
root@secret-env-pod:/# echo $DB_PASSWORD
Mypass123
root@secret-env-pod:/#

```

4. Create a Secret using a YAML file, mount it as a volume in a pod, and verify the specific Secret values are available as files.

Step 1: Create Secret YAML

```

apiVersion: v1
kind: Secret
metadata:
  name: database-secret
type: Opaque
stringData:
  username: admin
  password: MyPass123

```

Why did I use stringData?

Because it is easier for learning.

You don't need to manually convert:

admin

to Base64.

Kubernetes handles it.

Step 2: Create Secret & Check:

```
ubuntu@k8s-master:~$ vi database-secret.yaml
ubuntu@k8s-master:~$ kubectl apply -f database-secret.yaml
secret/database-secret created
ubuntu@k8s-master:~$ kubectl get secret database-secret
NAME                TYPE      DATA   AGE
database-secret     Opaque    2       10s
ubuntu@k8s-master:~$
```

Step 3: Create Pod

```
GNU nano 2.9.1
apiVersion: v1
kind: Pod
metadata:
  name: secret-volume-pod
spec:
  containers:
  - name: nginx
    image: nginx

    volumeMounts:
    - name: secret-volume
      mountPath: /etc/mysecret
      readOnly: true

  volumes:
  - name: secret-volume
    secret:
      secretName: database-secret
```

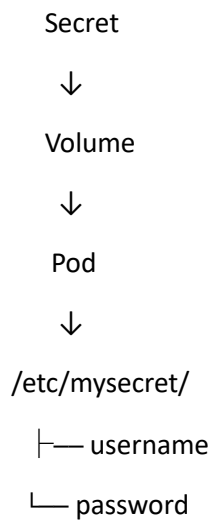
Step 4: Apply and Check

```
ubuntu@k8s-master:~$ nano secret-volume-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f secret-volume-pod.yaml
pod/secret-volume-pod created
ubuntu@k8s-master:~$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
configmap-env-pod   1/1     Running   0           47m
configmap-volume-pod 1/1     Running   0           30m
default-nginx       1/1     Running   1 (176m ago) 26h
secret-env-pod      1/1     Running   0           16m
secret-volume-pod   1/1     Running   0           12s
ubuntu@k8s-master:~$
```

Step 5: Check files

```
ubuntu@k8s-master:~$ kubectl exec -it secret-volume-pod -- /bin/bash
root@secret-volume-pod:/# ls /etc/mysecret
password  username
root@secret-volume-pod:/# cat /etc/mysecret/username
adminroot@secret-volume-pod:/# cat /etc/mysecret/password
MyPass123root@secret-volume-pod:/#
```

Flow:



5. Inject a ConfigMap as environment variables and a Secret as files into the same pod, ensuring both are accessible within the pod.

Step 1: Create ConfigMap

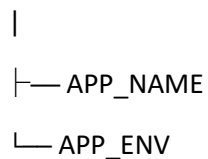
```
ubuntu@k8s-master:~$ kubectl create configmap app-settings \
  --from-literal=APP_NAME=MyApplication \
  --from-literal=APP_ENV=production
configmap/app-settings created
```

Step 2: Create Secret

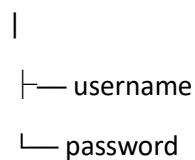
```
ubuntu@k8s-master:~$ kubectl create secret generic app-secret \
  --from-literal=username=admin \
  --from-literal=password=MyPass123
secret/app-secret created
ubuntu@k8s-master:~$
```

Now we have:

ConfigMap



Secret



Step 3: Create Pod

```
ubuntu@k8s-master:~$ nano config-secret-pod.yaml
```

```

GNU nano 8.7.1 config-secret-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: config-secret-pod
spec:
  containers:
    - name: nginx
      image: nginx

      envFrom:
        - configMapRef:
            name: app-settings

      volumeMounts:
        - name: secret-volume
          mountPath: /etc/app-secret
          readOnly: true

  volumes:
    - name: secret-volume
      secret:
        secretName: app-secret

```

Step 4: Apply

```

ubuntu@k8s-master:~$ kubectl apply -f config-secret-pod.yaml
pod/config-secret-pod created
ubuntu@k8s-master:~$

```

Check:

```

ubuntu@k8s-master:~$ kubectl get pod config-secret-pod
NAME             READY   STATUS    RESTARTS   AGE
config-secret-pod 1/1     Running   0           71s
ubuntu@k8s-master:~$

```

Step 5: Check ConfigMap

```

ubuntu@k8s-master:~$ kubectl exec -it config-secret-pod -- /bin/bash
root@config-secret-pod:/# echo $APP_NAME
MyApplication
root@config-secret-pod:/# echo $APP_ENV
production
root@config-secret-pod:/#

```

So ConfigMap is working as environment variables.

Step 6: Check Secret

Still inside the same Pod:

```

root@config-secret-pod:/# ls /etc/app-secret
password  username
root@config-secret-pod:/# cat /etc/app-secret/username
root@config-secret-pod:/# cat /etc/app-secret/passwordsword
MyPass123root@config-secret-pod:/#

```

So Secret is working as files.